



Loyola Marymount
University

CMSI 5350/EECE 5150 Machine Learning

Professor Lanyu Shang

Announcements

- Homework 2 due **this Friday 10/2**
 - 5-point bonus for early submission by 5 PM.
 - Don't forget to tag your solution to corresponding questions!
- PostCommit quiz **next Tuesday 10/6** during lecture
 - Bring your laptop!
 - 3 questions in 20 minutes

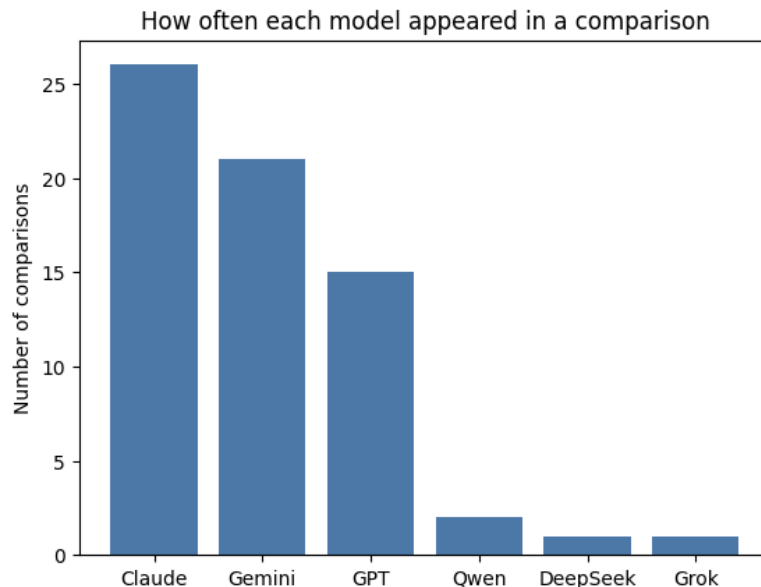
Homework 1 - Grading

- Weighted grade -> Brightspace
- Detailed feedback & raw score -> Gradescope
- Solution -> Brightspace

The screenshot displays the Brightspace user interface. On the left is a navigation sidebar with a search bar labeled 'Search Topics' and a magnifying glass icon. Below the search bar are five menu items: 'Overview' with a monitor icon, 'Bookmarks' with a bookmark icon, 'Course Schedule' with a calendar icon and a badge showing '2', 'Table of Contents' with a document icon and a badge showing '9', and 'Homework Solutions' with a document icon and a badge showing '1'. The 'Homework Solutions' item is highlighted with a blue background. The main content area on the right has the title 'Homework Solutions' followed by a dropdown arrow. In the top right corner of this area is a 'Print' button with a printer icon. Below the title is a 'Download' button with a download icon. A progress bar is shown with the text '0 % 0 of 1 topics complete'. At the bottom, there is a link 'CMSI 5350 - Homework 1 - Solution' with a dropdown arrow, and below it, a document icon and the text 'PDF document'.

Homework 1 - GenAI

- Which one was the most/least popular?
- Which one wined?
 - Claude vs Gemini: 15 vs 3
 - GPT vs Claude: 5 vs 4
 - GPT vs Gemini: 3 vs 2
- Try different pairs in HW2?

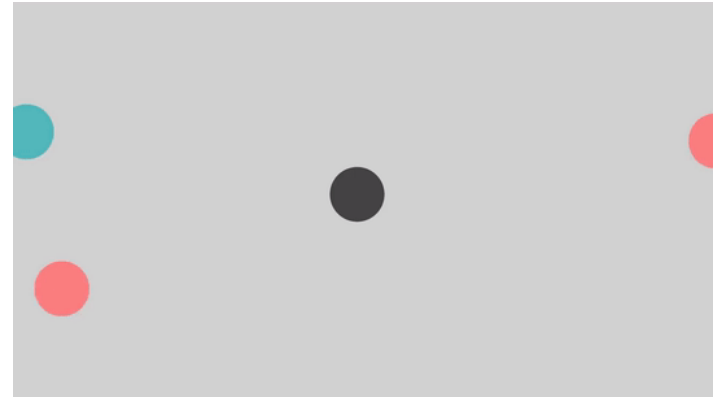
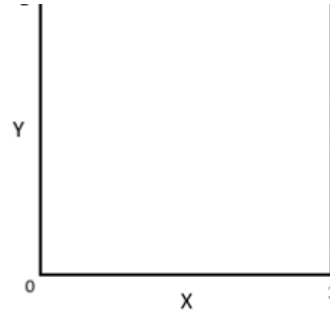


This lecture

- DT and KNN implementation
- Linear regression
 - Problem setting
 - Objective function
 - Learning / training
 - Gradient descent
 - Normal equation

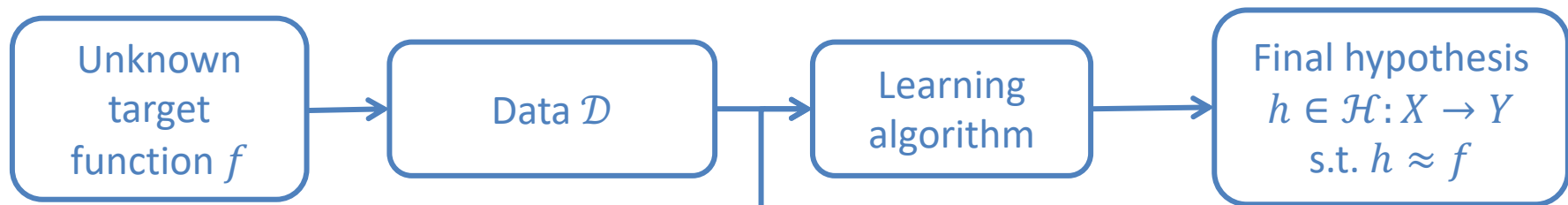
Last lecture

- Decision tree and KNN



ML problem settings

For decision tree?



Paired data

X, y

ID3, C4.5, CART

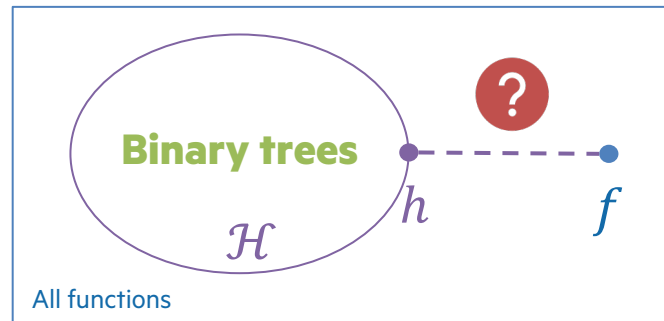
criterion: {"gini",
"entropy", "log_loss"}

Fitted tree

`clf.fit(X, y)`

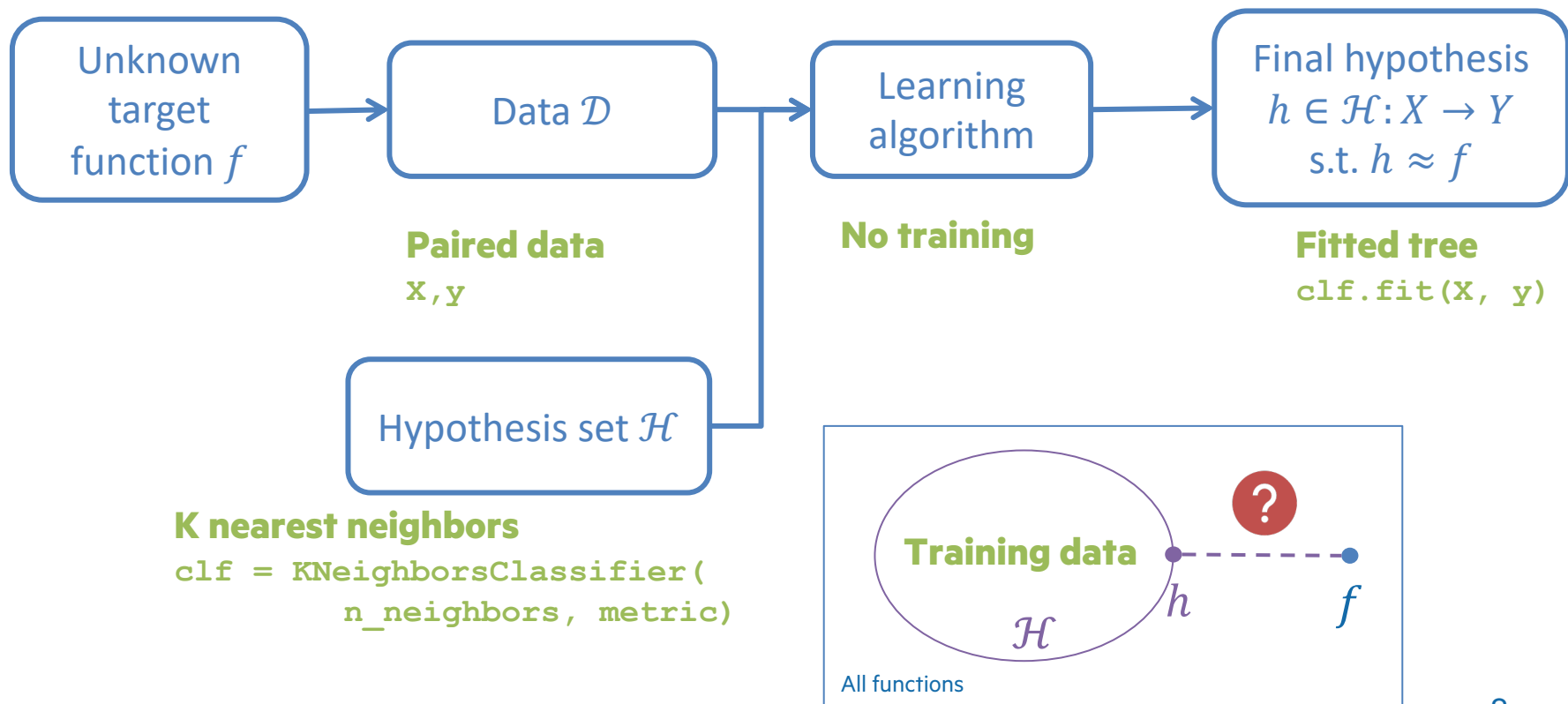
Binary trees

`clf = DecisionTreeClassifier()`



ML problem settings

For KNN?

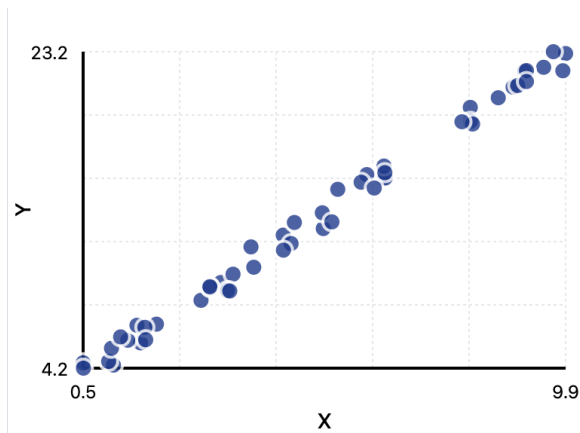


Colab Notebook

DECISION TREE & KNN

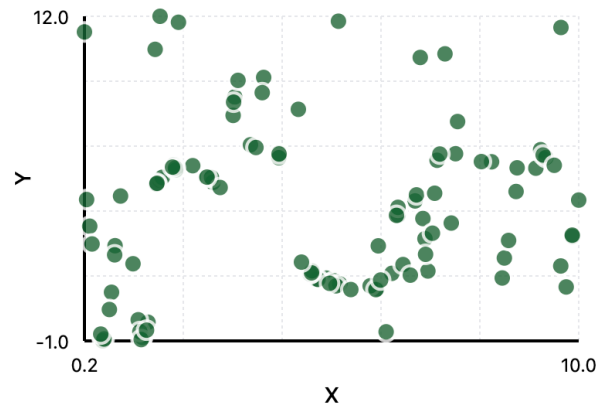
Non-parametric vs parametric

- Which model to choose? DT, KNN, or linear regression?



Parametric (e.g., linear regression)

- + Strong assumptions about data distribution
- + Few or fixed parameters (e.g., slope, intercept)
- + Faster prediction once trained

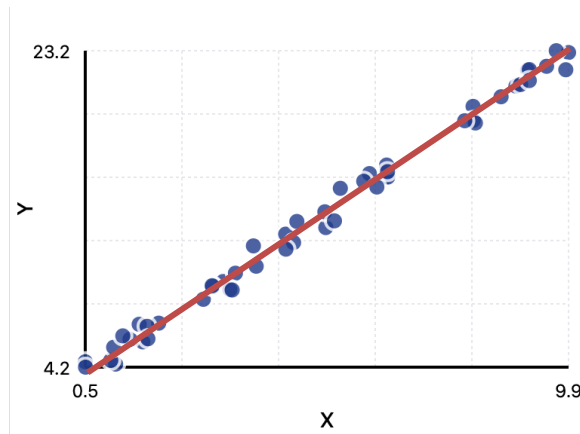


Non-Parametric (e.g., DT, KNN)

- + Complex, changing patterns
- + Local variations
- + No obvious functional form/flexible

Linear regression – problem settings

- Data $X = \{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(n)}\}$
 - Each $\mathbf{x}^{(i)} \in \mathbb{R}^d$ is a d-dimensional feature vector
- Labels $Y = \{y^{(1)}, y^{(2)}, \dots, y^{(n)}\}$
 - Each $y^{(i)} \in \mathbb{R}$
- Unknown target function $f: X \rightarrow Y$
- Hypothesis set $\mathcal{H} = \{h \mid h: X \rightarrow Y\}$
 - Each hypothesis h is a linear model
 - E.g, $y = \alpha x + \beta$



Linear regression

- Hypothesis

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_d x_d$$

h is parameterized by θ

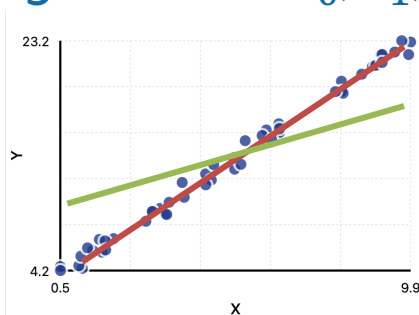
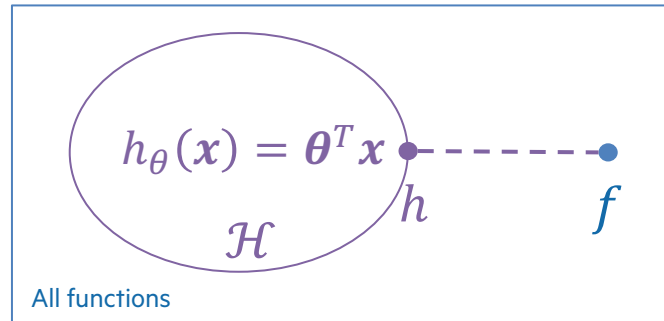
$$= \sum_{j=0}^{j=d} \theta_j x_j = \theta^T x$$

- Features $x = \langle x_0, x_1, x_2, \cdots, x_d \rangle$

- Parameters/weights $\theta = \langle \theta_0, \theta_1, \theta_2, \cdots, \theta_d \rangle$

- Goal

- $h \approx f$



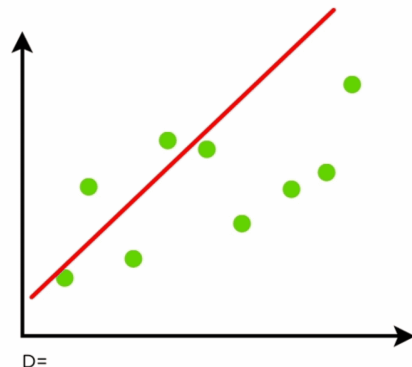
Which fitted line
is better?

Objective function

- Objective/cost function

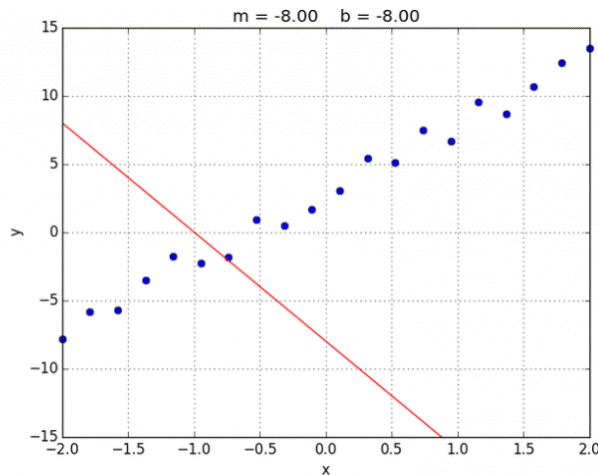
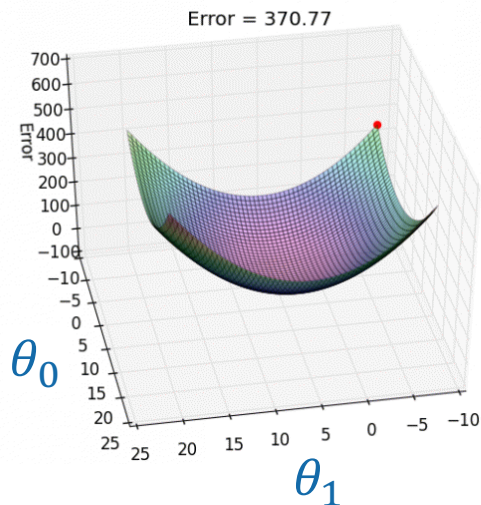
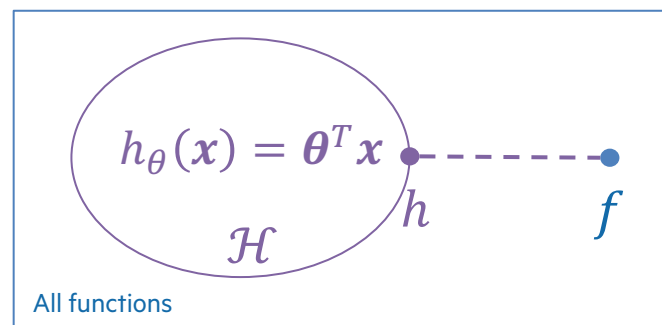
$$J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2 = \frac{1}{2n} \sum_{i=1}^n (\overset{\text{Predicted } y}{h_{\boldsymbol{\theta}}(\boldsymbol{x}^{(i)})} - \overset{\text{Ground truth } y}{y^{(i)}})^2$$

- The **squared error** of the difference between every sample in the **training** set
- Learning objective
 - **Minimize** objective function $J(\boldsymbol{\theta})$
 - Find/search optimal $\boldsymbol{\theta}^*$ s.t. $\boldsymbol{\theta}^* = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} J(\boldsymbol{\theta})$



Learning / training LR

- Finding/searching optimal θ^* , how?



- Start w/ an initial value for θ
- Until we reach minimum:
 - Choose new value for θ to reduce $J(\theta)$

Approach 1: gradient descent

- Example: <https://www.i-am.ai/gradient-descent.html>
- Gradient
 - Vector of partial derivatives
 - Points in the direction of **maximum increase** of the function
- Gradient descent
 - Points in the direction of **steepest descent** to minimize

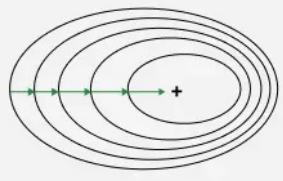
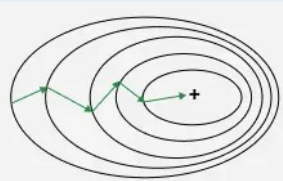
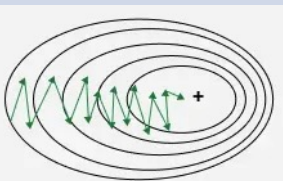
Gradient descent

α is the learning rate (e.g., 0.05)

- Gradient descent in each dimension: $\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\boldsymbol{\theta})$
- Gradient descent along all dimensions: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \nabla J(\boldsymbol{\theta})$
- -> Worksheet
- The algorithm

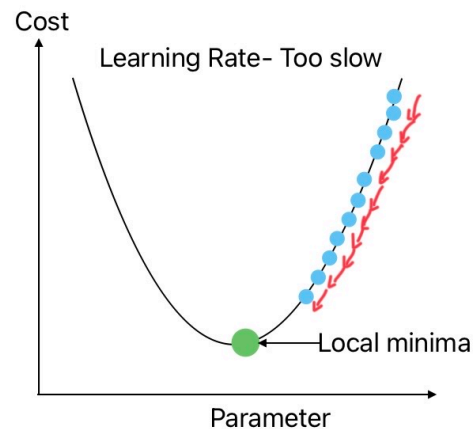
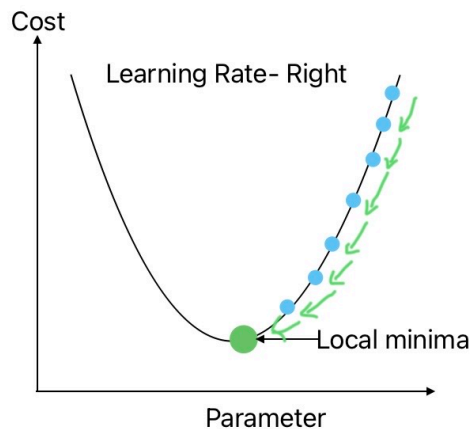
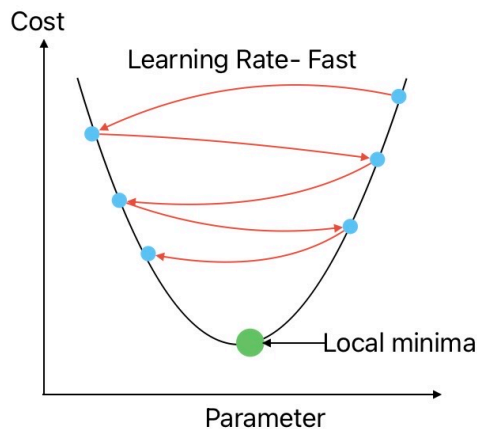
```
for i = 1 to max_iteration:
    J(θ) = cost_function(θ) (on all data samples)
    computer ∇J(θ)
    θ = θ - α * ∇J(θ)
    IF |∇J(θ)| < tolerance: break
end for
```

Variants of gradient descent

Method	Samples per Update	Efficiency	Escaping Local Minima	Convergence	Visualization
(Batch) Gradient Descent	All samples	Slowest	Poor	Smooth, stable	
Mini-Batch Gradient Descent	k samples (32-512)	Medium	Moderate	Moderately smooth	
Stochastic Gradient Descent	1 sample	Fastest	Good (noise helps)	Noisy, oscillating	

Learning rate α

- Hyperparameter
 - Usually a float: 0.1, 0.01, 0.001



Approach 2: normal equation

- A direct, closed-form solution

For 1 sample, let $\boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{bmatrix}$ and $\mathbf{x} = \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_d \end{bmatrix}$  $h_{\boldsymbol{\theta}}(\mathbf{x}) = \sum_{j=0}^d \theta_j x_j = \boldsymbol{\theta}^T \mathbf{x}$

For n sample,

$$X = \begin{bmatrix} (x^{(1)})^T \\ \vdots \\ (x^{(i)})^T \\ \vdots \\ (x^{(n)})^T \end{bmatrix} = \begin{bmatrix} 1 & x_1^{(1)} & \cdots & x_d^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(i)} & \cdots & x_d^{(i)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n)} & \cdots & x_d^{(n)} \end{bmatrix}$$

n samples x (d+1) features


$$h_{\boldsymbol{\theta}}(\mathbf{X}) = \mathbf{X} \boldsymbol{\theta}$$

 $n \times 1$  $n \times (d+1)$  $(d+1) \times 1$

Matrix-Vector form of cost function

- Let $\mathbf{y} = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(n)} \end{bmatrix}$

$$\mathbf{X}\boldsymbol{\theta} - \mathbf{y} = \begin{bmatrix} (\mathbf{x}^{(1)})^T \boldsymbol{\theta} \\ \vdots \\ (\mathbf{x}^{(n)})^T \boldsymbol{\theta} \end{bmatrix} - \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(n)} \end{bmatrix} = \begin{bmatrix} h_{\boldsymbol{\theta}}(\mathbf{x}^{(1)}) - y^{(1)} \\ \vdots \\ h_{\boldsymbol{\theta}}(\mathbf{x}^{(n)}) - y^{(n)} \end{bmatrix}$$

$$\mathbf{z}^T \mathbf{z} = \sum_j z_j^2 \quad \Rightarrow \quad \frac{1}{2} (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})^T (\mathbf{X}\boldsymbol{\theta} - \mathbf{y}) = \frac{1}{2} \sum_{i=1}^n \left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)} \right)^2 = J(\boldsymbol{\theta})$$

Solve for θ analytically

- $\theta^* = \arg \min_{\theta} J(\theta)$ attained when $\nabla_{\theta} J(\theta) = 0$

$$\begin{aligned} J(\theta) &= \frac{1}{2} (\mathbf{X}\theta - \mathbf{y})^T (\mathbf{X}\theta - \mathbf{y}) \\ &= \frac{1}{2} \left[\theta^T \mathbf{X}^T \mathbf{X} \theta - \theta^T \mathbf{X}^T \mathbf{y} - \mathbf{y}^T \mathbf{X} \theta + \mathbf{y}^T \mathbf{y} \right] \\ &= \frac{1}{2} \left[\theta^T \mathbf{X}^T \mathbf{X} \theta - 2\theta^T \mathbf{X}^T \mathbf{y} + \mathbf{y}^T \mathbf{y} \right] \end{aligned}$$

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \mathbf{X}^T \mathbf{X} \theta - \mathbf{X}^T \mathbf{y} = 0 \\ \mathbf{X}^T \mathbf{X} \theta &= \mathbf{X}^T \mathbf{y} \end{aligned} \quad \Rightarrow \quad \theta^* = \left(\mathbf{X}^T \mathbf{X} \right)^{-1} \mathbf{X}^T \mathbf{y}$$

Gradient descent vs. normal equation

- Gradient descent
 - requires multiple iterations
 - need to choose α
 - works well when d is large
 - can support online learning
- Normal equation
 - non-iterative
 - no need for α
 - slow if d is large